API REST en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. L'API Rest : principe de fonctionnement 3

A. L'API Rest : principe de fonctionnement 3

B. Exercice : Quiz 5

III. Créer une API Rest avec Flask 6

A. Créer un serveur Rest simple 6

B. Interroger et tester notre serveur 8

C. Exercice : Quiz 13

IV. Essentiel 14

V. Auto-évaluation 14

A. Exercice 14

B. Test 14

Solutions des exercices 15

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python, Bases de Flask, Requêtes http

Contexte

Vous est-il déjà arrivé de vous demander comment des entités logicielles différentes (et même parfois éloignées) pouvaient communiquer entre elles ?

Il existe un moyen simple, standardisé et extrêmement réputé : une API Rest. L'API Rest n'est rien d'autre qu'une interface, qui permet d'échanger des informations avec une application.

Dans ce cours, nous allons découvrir ce qu'est réellement une API Rest, et nous allons voir à quel point ce type d'interface peut être répandu à l'heure actuelle.

Nous verrons également comment mettre en place ce type d'interface sur un exemple concret, et vous constaterez que derrière ce nom mystérieux se cache un concept extrêmement simple.

II. L'API Rest : principe de fonctionnement

A. L'API Rest : principe de fonctionnement

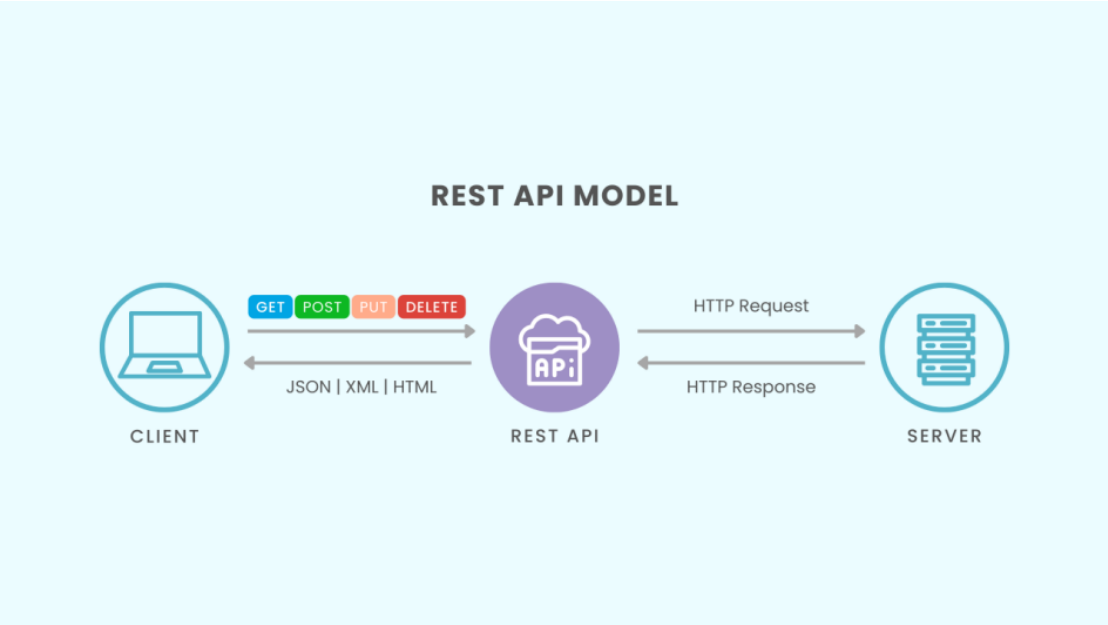
De manière générale, une API est une interface qui permet d'échanger des informations avec un serveur. Plus précisément, les termes « *API* » et « *Rest* » signifient :

- **API = Application Programming Interface**

  Il s'agit tout simplement d'une spécification qui décrit la façon dont il faut s'y prendre pour échanger des informations avec le serveur.

- **Rest = Il s'agit du diminutif de « *Restful* ».**

  En français, cela signifie « *sans état* ». Ce nom vient du fait que pour une API « *Rest* », les échanges se font à travers des messages qui ne stockent pas l'état de la connexion entre l'utilisateur et le serveur. Autrement dit, toutes les requêtes se ressemblent, puisqu'elles contiennent à chaque fois l'ensemble des informations relatives à la connexion entre les entités (adresse, login, mot de passe, etc.).



Source : SKIPLEVEL[1]

Nous verrons que la notion d'API Rest est intimement liée à de nombreux sujets que nous avons déjà eu l'occasion de couvrir lors des cours précédents. Par exemple, les requêtes http ou encore les bases du framework Flask. Nous verrons même que sans le savoir, nous avons déjà implémenté une API Rest à l'aide de Flask !

Avant de passer dans la pratique, voyons un peu ce qui se cache exactement derrière ce concept d'API Rest. Qu'est ce qui la caractérise ? Et dans quelle mesure elle est employée à l'heure actuelle ?

D'un point de vue technique, l'API Rest n'est pas le seul moyen de communiquer avec un serveur distant. En effet, selon l'architecture et les contraintes systèmes, plusieurs solutions s'offrent à vous.

Citons quelques exemples :

- **Le protocole MQTT** permet de relier un serveur à un « *distributeur* » (on appelle ça un « *broker* »). Les clients (les utilisateurs) doivent notifier à ce broker qu'ils souhaitent être notifiés si certaines variables sont mises à jour. Ainsi, dès que le serveur publie une nouvelle variable, les clients sont notifiés et reçoivent la nouvelle valeur,
- **Les WebSockets** sont un moyen de communication direct entre deux entités logicielles. Dans un websocket, un canal de communication direct est établi à travers une connexion TCP. Un échange de données n'est possible que si le canal est préexistant, ce qui implique naturellement des mécanismes de connexion, et de handshake notamment. Généralement, ce protocole est utilisé pour des besoins bien spécifiques, qui requièrent une persistance de connexion, ainsi que des performances élevées (par exemple, du streaming)
- **Le protocole SOAP (Simple Object Access Protocol)** est relativement proche de l'API Rest, bien que plus ancien que ce dernier. Il est aussi beaucoup plus complexe et moins performant. Le protocole SOAP consiste à échanger des données en les formatant sous forme de fichier XML. A l'inverse de l'API Rest, le protocole SOAP est extrêmement contraint au niveau de son template, ce qui le rend finalement peu flexible.

Dans les années 2000, un certain Roy Fielding réalise une dissertation dont le sujet était de trouver une solution simple pour que deux serveurs distants puissent communiquer entre eux. De cette dissertation est né l'API Rest, qui est caractérisée par plusieurs critères :

1. **Interface standard** : les méthodes utilisées doivent se limiter à des requêtes http (GET, PUT, POST, DELETE). De ces requêtes, il faut toujours pouvoir récupérer une réponse nous indiquant le statut de la requête.
2. **Sans état (Stateless)** : cela signifie qu'il n'y a pas de mémorisation de l'état de connexion. Chaque requête doit être « *autoportante* », c'est-à-dire qu'elle ne doit pas nécessiter de pré-connexion pour pouvoir être achevée.
3. **Lien client-serveur** : la liaison s'établit entre un client (qui interroge le serveur) et un serveur
4. **Cacheable (mise en cache possible)** : cela signifie que puisque chaque requête est « *autoportante* », le contenu d'une réponse peut être stocké (pour être ré-utilisé plus tard). Par exemple, si l'application utilisateur demande une variable au serveur, et que cette variable n'est pas censée changer, le serveur en notifiera le client lors de la réponse, de manière à ce que le client puisse mémoriser la réponse et ne plus redemander le même contenu plus tard.

1 https://scenari-author.studi.fr/~~static/fr-FR/home.xhtml?~1jlzn#(wsp'skZn5aR2m8PEpNJNSMrwHJ'srcRef'id%3AGWBbHRlZ7MCTa9B8k3eRTn')

Tous droits réservés à STUDI - Reproduction interdite

Les API Rest sont tellement réputées que plusieurs grandes sociétés offrent ce type d'interface pour exposer leurs services. L'accès à ces interfaces est parfois rendu payant. Le gros avantage est de pouvoir utiliser ces accès API directement dans vos applications Web. Dans ce cas, vous êtes client, et eux, le serveur.

Quelques exemples de services offrant des accès API :

1. **ChatGPT**
2. **Google Maps**
3. **Github**

Ainsi, moyennant (souvent) un abonnement payant, votre application peut directement interroger ces services. Chaque service met à disposition ce qu'on appelle des « *endpoints* », c'est-à-dire des fonctionnalités que vous pouvez interroger.

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Que signifie l'acronyme API ?

- ○ Application Power Interface
- ○ All Program Information
- ○ Application Programming Interface

Question 2

Que signifie le terme Restful ?

- ○ Que l'application ne fait rien
- ○ Que l'application reste connectée en permanence
- ○ Que l'état de connexion n'est pas mémorisé

Question 3

L'API Rest est le seul moyen pour communiquer avec un serveur distant :

- ○ Vrai
- ○ Faux

Question 4

L'API Rest est extrêmement utilisée.

- ○ Vrai
- ○ Faux

Question 5

Une API Rest utilise des requêtes HTTP.

- ○ Vrai
- ○ Faux

Tous droits réservés à STUDI - Reproduction interdite

III. Créer une API Rest avec Flask

A. Créer un serveur Rest simple

Comme nous l'avons dit précédemment, il est possible de réaliser un serveur intégrant une API Rest à l'aide du framework Flask.

Fonctionnellement, si vous avez retenu ce que nous avons vu dans la 1ère partie « *L'API Rest : principe de fonctionnement* », c'est extrêmement simple : il suffit d'exposer nos variables ou nos méthodes à des routes, associées à des commandes http, accessibles par le client distant. Voilà un bref résumé de ce en quoi consiste l'implémentation d'un serveur Rest.

Cela peut tout de même paraître un peu flou, voyons un exemple concret.

Exemple

Dans cet exemple, nous allons développer un serveur REST qui gère une base de données d'utilisateurs.

Nous souhaitons établir une API Rest, qui permet à un utilisateur distant de réaliser les actions suivantes :

- Récupérer la liste des utilisateurs
- Ajouter un utilisateur
- Supprimer un utilisateur

Commençons par créer un projet Flask, et créons une base de données d'utilisateurs très simple (nous utiliserons tout simplement un array pour cet exemple) :

```
1 from flask import Flask, request, jsonify
2 import json
3
4 app = Flask(__name__)
5
6 users = [
7     {'id': 1, 'user': 'Jean'},
8     {'id': 2, 'user': 'Arnaud'},
9     {'id': 3, 'user': 'Charles'},
10    {'id': 4, 'user': 'Marie'}
11 ]
```

On peut déjà constater qu'il y a plusieurs librairies qui sont importées. Flask, évidemment, Request pour gérer les requêtes http, et Jsonify qui permet de créer un objet JSON à partir d'objets élémentaires Python. Généralement, les données échangées à l'aides d'une API Rest est formatées dans un objet JSON.

Ensuite, nous définissons un tableau de tuples : 4 utilisateurs sont mentionnés dans le tableau.

Implémentons l'ensemble des fonctions qui ont été invoquées plus haut :

```
1 @app.route('/users', methods=['GET'])
2 def getUsersList():
3     return jsonify(users)
```

Ici, on définit une route « */users* », qui, associée avec la méthode http 'GET', va renvoyer un objet JSON qui contient le tableau d'utilisateurs « *users* ».

```
1 @app.route('/users', methods=['POST'])
2 def createNewUser():
3     newUser = {'id' : 0, 'user': 'username'}
4     newUser['user'] = (request.args.get('user'))
5     print(newUser)
6     newUser['id'] = len(users)+1
7     print(newUser)
8     users.append(newUser)
```


Tous droits réservés à STUDI - Reproduction interdite

```
    return 'Success', 201
```

Dans ce cas, nous créons une route /users, mais associée à la méthode POST.

Nous créons d'abord un nouveau tuple initialisé à « *id = 0* » et « *user = username* ».

Ensuite, nous récupérons le contenu de la requête grâce à request.args.get().

On complète alors l'ID de ce nouvel utilisateur, en récupérant simplement la longueur de notre tableau d'utilisateurs existant, et en incrémentant de 1.

Finalement, on ajoute (append) ce nouvel utilisateur à notre tableau d'utilisateurs.

Une fois terminé, on renvoie un code 201 (code OK), ainsi que le message « *success* ».

Enfin, pour la méthode delete :

```
@app.route('/users', methods=['DELETE'])
def removeUser():
    idToRemove = int(request.args.get('id'))
    for user in users:
        if (int(user['id']) == idToRemove):
            users.pop(users.index(user))
    return 'Success', 200
```

Dans ce cas, on récupère l'id de l'utilisateur à supprimer, ensuite, on parcourt notre liste d'utilisateur à la recherche de celui dont l'id correspond.

Une fois qu'on a trouvé cet utilisateur, on le supprime de la liste (pop()).

Finalement, on renvoie un code 200 (success).

Tous droits réservés à STUDI - Reproduction interdite

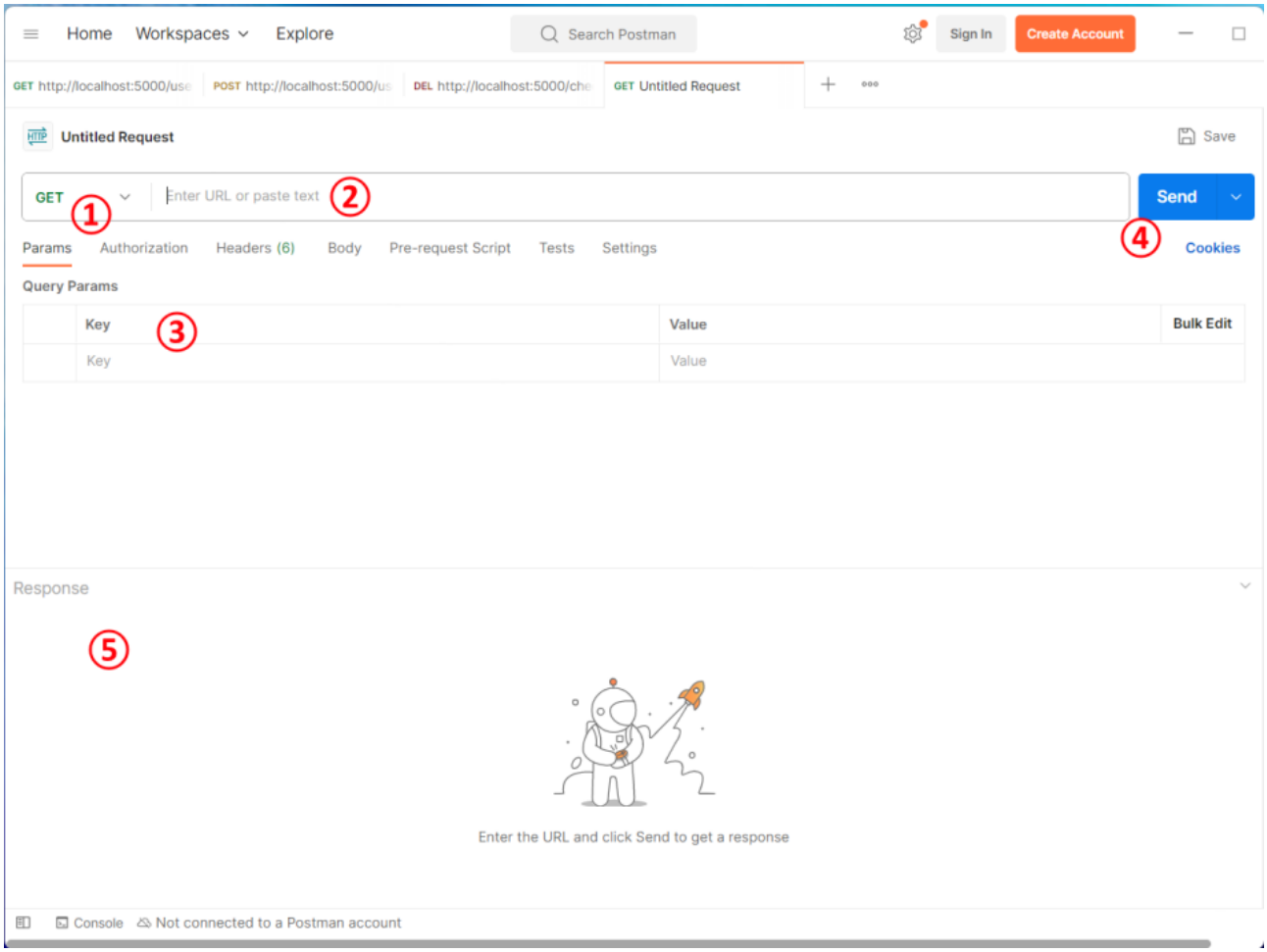
B. Interroger et tester notre serveur

Pour interroger et tester notre implémentation, il va nous falloir un client capable de réaliser des requêtes http.

Pour cela, il y a plusieurs solutions, dont notamment le client « *Postman* ».

Postman est un client http gratuit, qu'il est possible de télécharger au lien suivant : Downloads[1]

Une fois que Postman est installé sur votre machine, vous le démarrez et obtenez l'écran suivant :



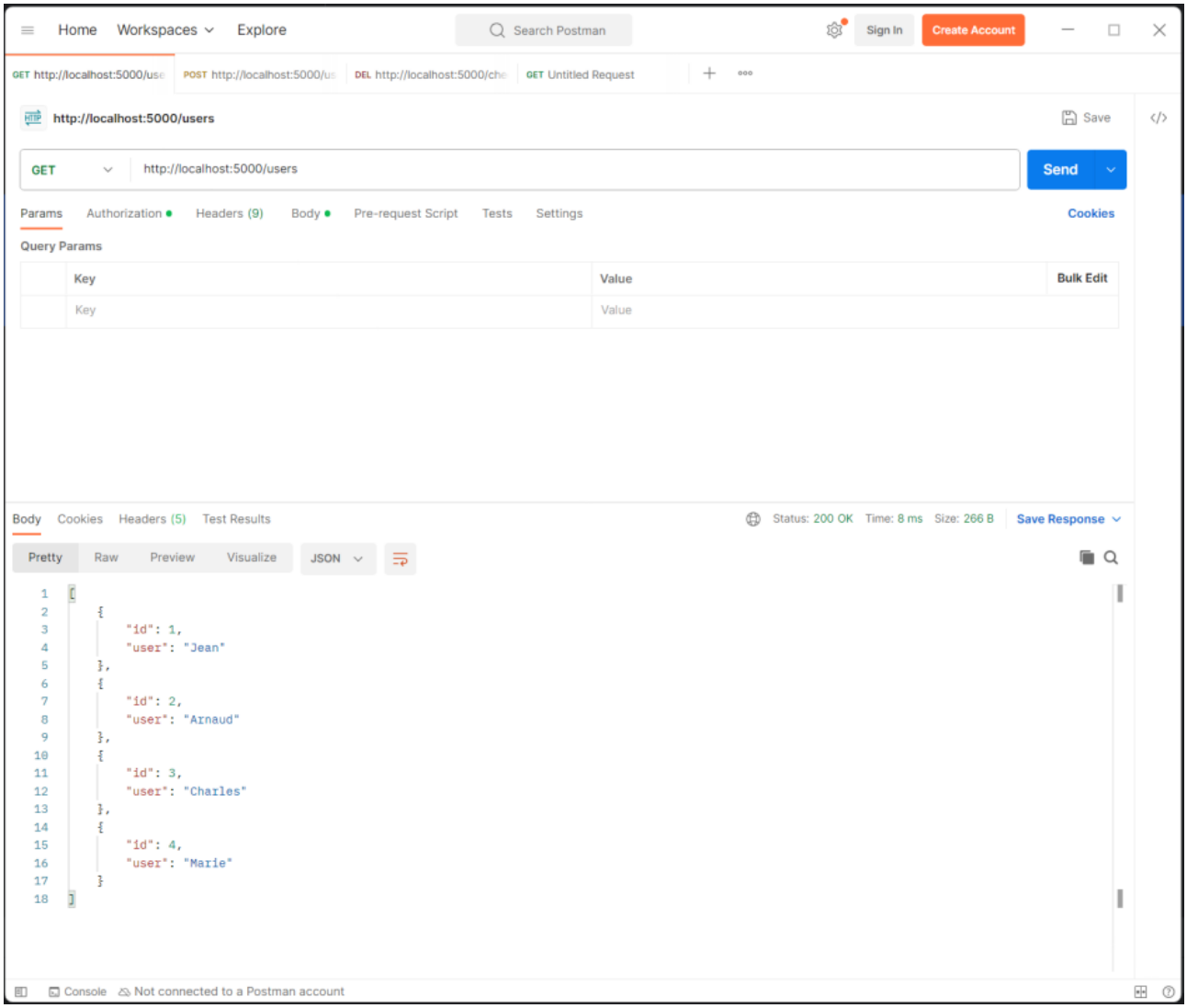
Les éléments importants sont annotés sur l'image et décrits ci-dessous :

1. Il s'agit de spécifier le type de requête http (par exemple GET, DELETE, etc.)
2. Insérez l'adresse. Par exemple http://127.0.0.1:5000[2] dans notre cas
3. Le cas échéant, il est possible de spécifier des paramètres dans la requête
4. Cliquez ensuite sur le bouton « *Send* »
5. Le résultat s'affiche dans cette zone de réponse

1 https://www.postman.com/downloads/
2 http://127.0.0.1:5000/

Tous droits réservés à STUDI - Reproduction interdite

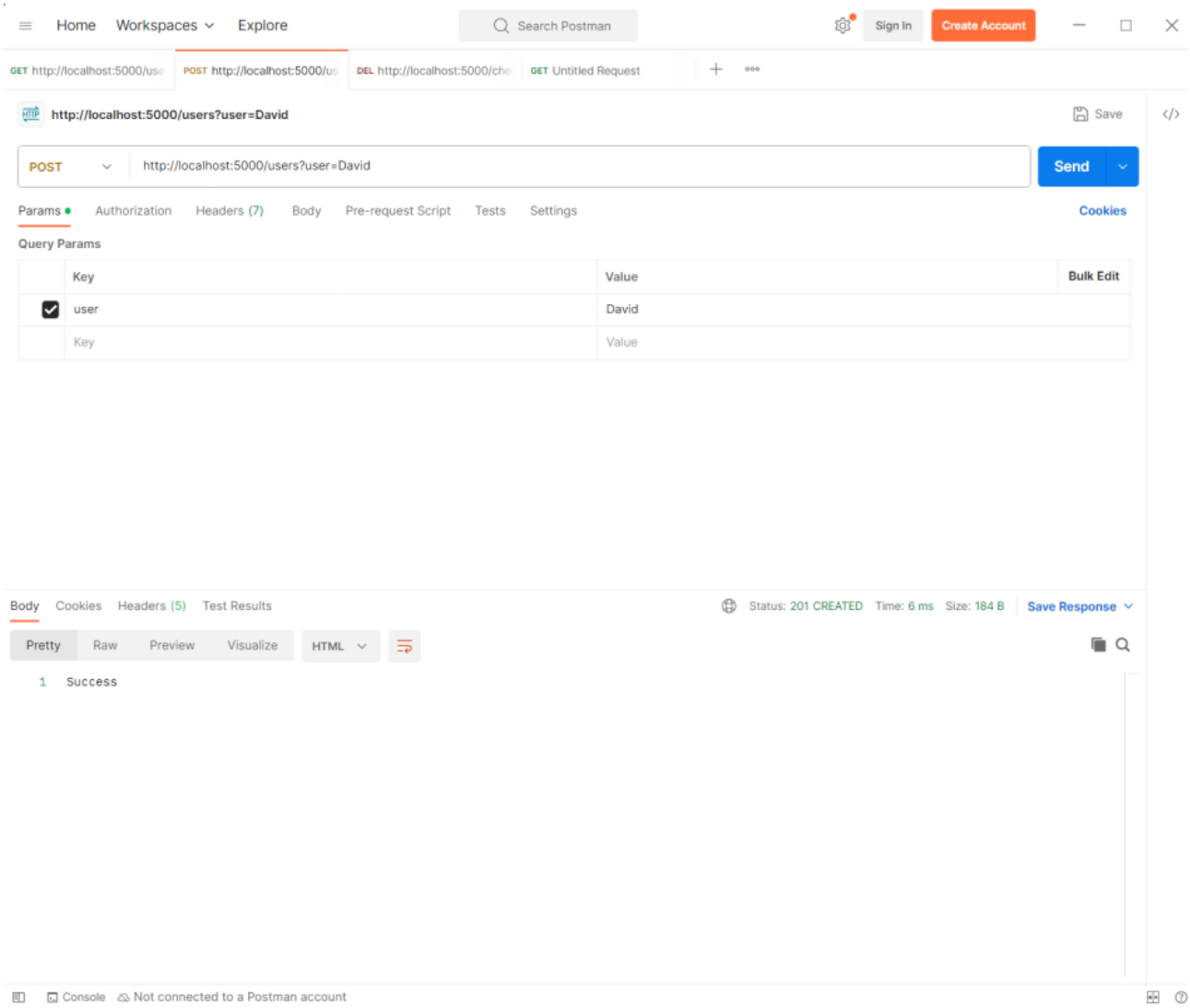
Dans notre exemple, pour récupérer la liste des utilisateurs, il suffit de faire une requête GET sur l'adresse « */users* » :



On constate qu'on récupère bien la liste de nos utilisateurs.

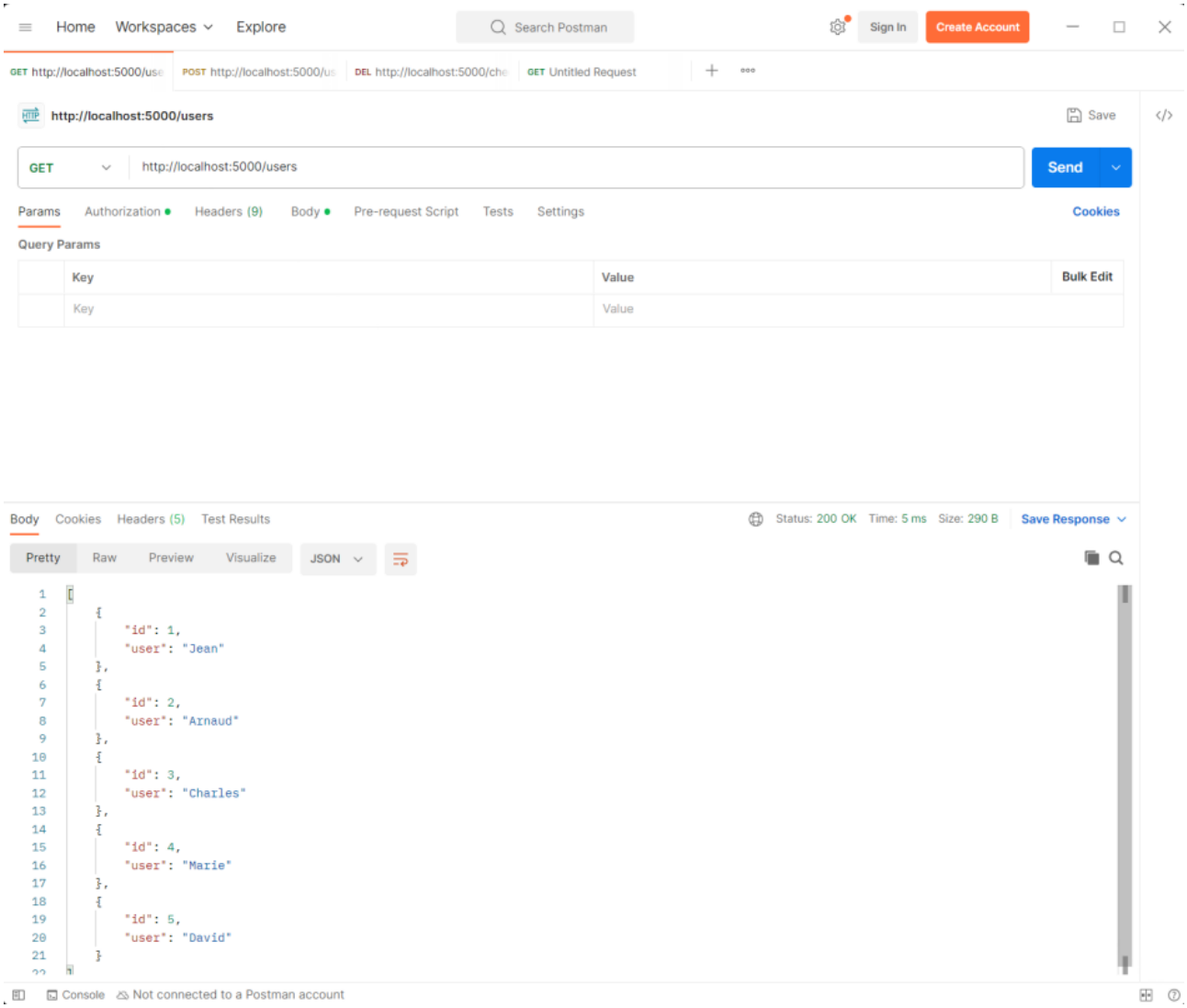
Tous droits réservés à STUDI - Reproduction interdite

Pour ajouter un utilisateur, il suffit de préciser en paramètres, le nom de cet utilisateur :



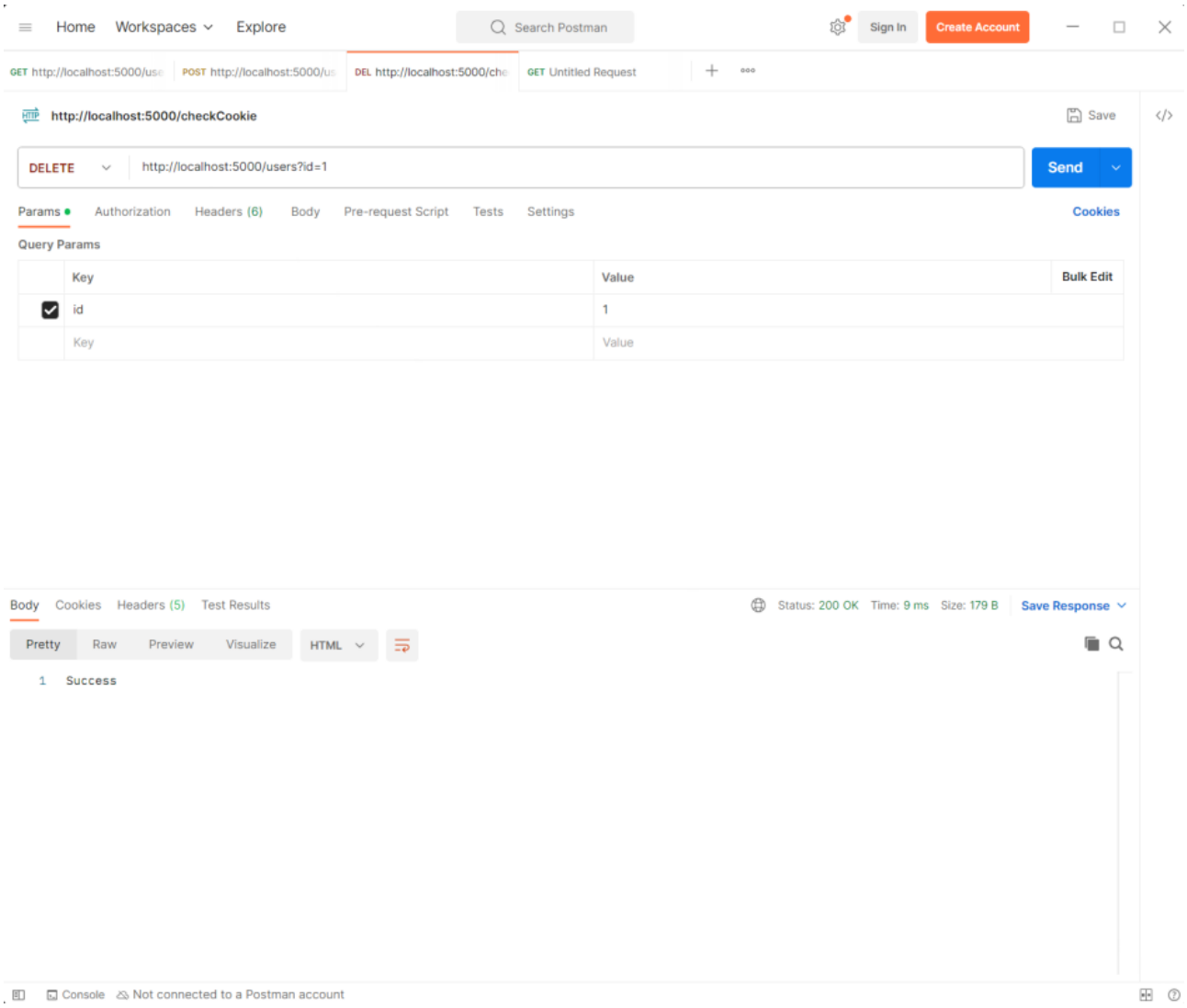
Tous droits réservés à STUDI - Reproduction interdite

On peut ainsi vérifier, avec une nouvelle requête GET sur « */users* », que notre utilisateur a bien été ajouté :



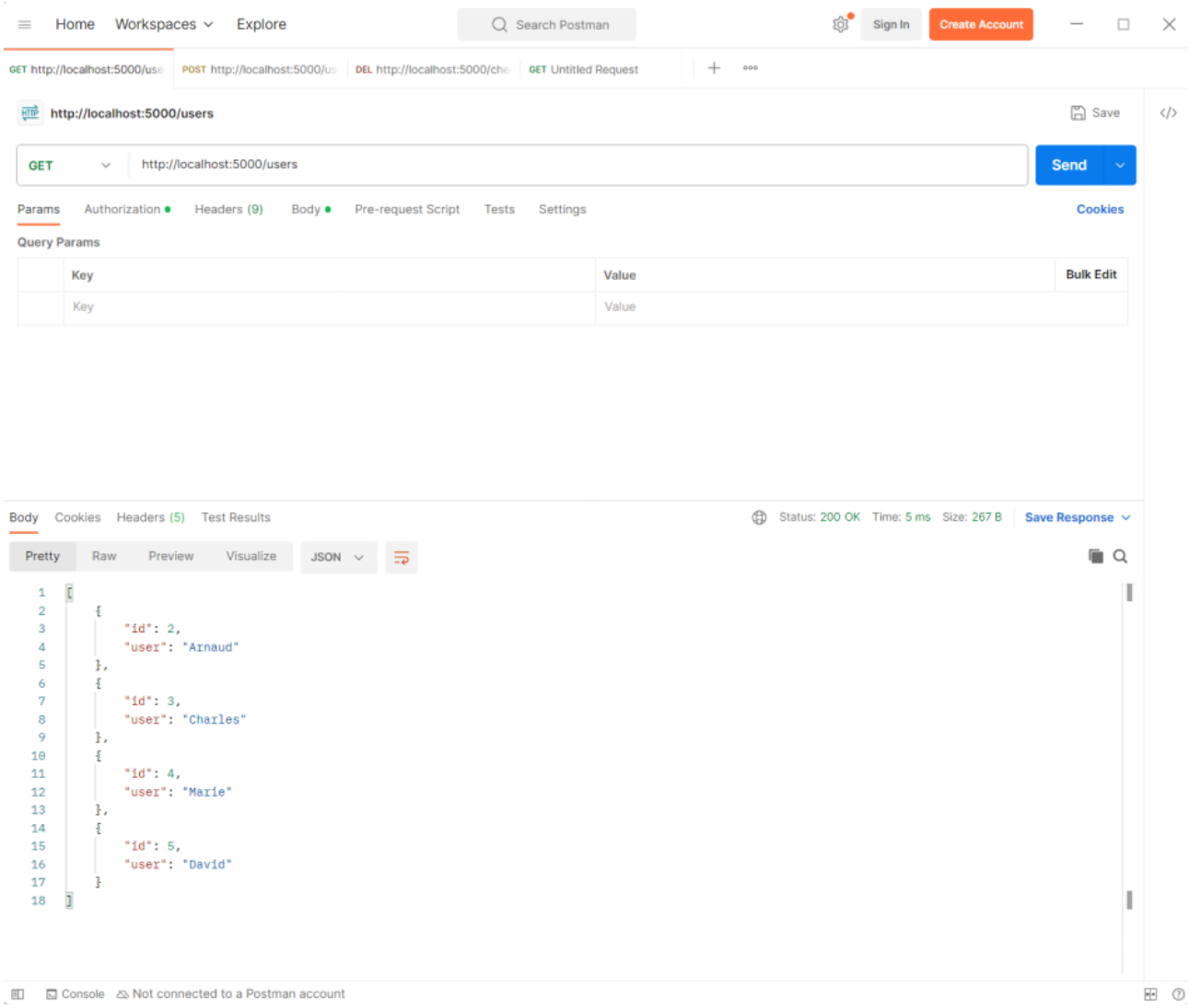
Tous droits réservés à STUDI - Reproduction interdite

Enfin, pour supprimer un utilisateur, il faut spécifier l'ID de ce dernier et préciser qu'il s'agit d'une opération « *DELETE* » :



Tous droits réservés à STUDI - Reproduction interdite

À nouveau, on peut vérifier que l'utilisateur avec l'ID 1 a bien été supprimé en récupérant la liste des utilisateurs :



C. Exercice : Quiz

[solution n°2 p.18]

Question 1

Flask permet de réaliser des serveurs avec une API Rest.

- ○ Vrai
- ○ Faux

Question 2

Flask est la seule alternative pour réaliser des serveurs Rest :

- ○ Vrai
- ○ Faux

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Pour réaliser une API Rest, il faut traiter les requêtes en fonction de...

- ○ Leur adresse (endpoint) et la méthode http utilisée
- ○ La variable demandée dans la requête
- ○ Leur adresse (endpoint), mais pas la méthode car il s'agit toujours d'une commande GET

Question 4

À quoi sert l'outil Postman ?

- ☐ À réaliser des requêtes http locales
- ☐ À réaliser des serveurs Rest
- ☐ À contrôler que notre serveur réagit correctement

Question 5

Dans une réponse à une requête http, que signifie le code 200 ?

- ○ Erreur : adresse introuvable
- ○ Commande correctement exécutée
- ○ Erreur : pas le droit d'accès à cette ressource

IV. Essentiel

Une API Rest n'est finalement rien de très complexe, une fois qu'on maitrise correctement les requêtes http et les bases du framework Flask.

En utilisant simplement les méthodes de routage de Flask, il est ainsi possible de créer un serveur Rest en seulement quelques lignes de code.

Dans ce cours, nous avons vu à quel point les API Rest étaient utiles (et utilisées) à l'heure actuelle. Nous avons parcouru un exemple pratique, et avons appris à tester notre serveur Rest à l'aide de l'outil Postman, qui permet de générer des requêtes http de tout type.

De manière générale, une API Rest permet d'ouvrir et de rendre disponible nos variables et nos méthodes au monde extérieur. Les notions à maitriser sont extrêmement simples, de par l'architecture (stateless) des échanges rendus possibles par une API Rest.

Bien entendu, même si nous avons utilisé le framework Flask (notamment pour le routage), d'autres frameworks proposent les mêmes fonctionnalités, par exemple Django et son module de routage est totalement équivalent.

V. Auto-évaluation

A. Exercice

Question

[solution n°3 p.19]

Nous avons réalisé un exemple de serveur Rest dans le cadre de ce cours. Etendez cet exemple en ajoutant la possibilité de récupérer le nom d'un utilisateur sur base de son ID. Par exemple, en entrant l'ID « *1* », on souhaite récupérer la réponse « *Jean* ».

B. Test

Exercice 1 : Quiz

[solution n°4 p.19]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la différence entre Websocket et API Rest ?

- ☐ Il n'y a pas de différence, c'est le même principe de fonctionnement
- ☐ Websocket n'est pas stateless
- ☐ Websocket est souvent plus rapide

Question 2

Qu'est ce qu'un '*endpoint*' ?

- ○ Une méthode HTTP
- ○ Une adresse spécifique sur laquelle une requête est réalisée
- ○ Le nom d'un protocole de communication

Question 3

Postman permet de faire des requêtes http de type GET uniquement :

- ○ Vrai
- ○ Faux

Question 4

Que signifie la notion de mise en cache dans un serveur Rest ?

- ○ La possibilité de masquer les adresses IP lors d'un échange de données
- ○ Il s'agit d'une caractéristique propre à l'architecture Rest, qui permet de donner une certaine persistance aux données récupérées du serveur
- ○ Il s'agit de la possibilité d'échanger des données sans maintenir une connexion persistante

Question 5

Flask est le seul framework permettant de réaliser des serveurs Rest.

- ○ Vrai
- ○ Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 Solution n°1

Question 1

Que signifie l'acronyme API ?

- ○ Application Power Interface
- ○ All Program Information
- ◉ Application Programming Interface

API signifie « *Application Programming Interface* ». Il s'agit d'une définition des interfaces d'une application serveur.

Question 2

Que signifie le terme Restful ?

- ○ Que l'application ne fait rien
- ○ Que l'application reste connectée en permanence
- ◉ Que l'état de connexion n'est pas mémorisé

Restful signifie qu'il n'y a pas d'échange de status de connexion. Autrement dit, l'état de connexion n'est pas mémorisé ni maintenu.

Question 3

L'API Rest est le seul moyen pour communiquer avec un serveur distant :

- ○ Vrai
- ◉ Faux

C'est faux, il existe d'autres moyens, plus ou moins intéressants en fonction des contraintes en place.

Question 4

L'API Rest est extrêmement utilisée.

- ◉ Vrai
- ○ Faux

Oui, et la plupart des grandes sociétés offrant des services Web ont d'ailleurs une API Rest.

Question 5

Une API Rest utilise des requêtes HTTP.

- ◉ Vrai
- ○ Faux

Exactement, une API Rest se base sur des requêtes http de type GET et POST par exemple.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 13 Solution n°2

Question 1

Flask permet de réaliser des serveurs avec une API Rest.

- (●) Vrai
- ( ) Faux

C'est vrai, Flask permet de réaliser une API Rest.

Question 2

Flask est la seule alternative pour réaliser des serveurs Rest :

- ( ) Vrai
- (●) Faux

Même si nous n'avons vu que Flask dans le cadre de ce cours, il existe évidemment d'autres alternatives pour pouvoir mettre en place votre serveur Rest.

Question 3

Pour réaliser une API Rest, il faut traiter les requêtes en fonction de...

- (●) Leur adresse (endpoint) et la méthode http utilisée
- ( ) La variable demandée dans la requête
- ( ) Leur adresse (endpoint), mais pas la méthode car il s'agit toujours d'une commande GET

En effet, il s'agit de traiter l'adresse utilisée, ainsi que la méthode (par exemple GET, POST, DELETE, etc.). Il peut y avoir plusieurs commandes possibles sur un seul endpoint, d'où l'importance de le préciser.

Question 4

À quoi sert l'outil Postman ?

- [x] À réaliser des requêtes http locales
- [ ] À réaliser des serveurs Rest
- [x] À contrôler que notre serveur réagit correctement

Les deux propositions (1 et 3) sont valables. Postman permet de réaliser des requêtes (locales ou distantes), et nous permet ainsi de valider le bon fonctionnement de notre serveur.

Question 5

Dans une réponse à une requête http, que signifie le code 200 ?

- ( ) Erreur : adresse introuvable
- (●) Commande correctement exécutée
- ( ) Erreur : pas le droit d'accès à cette ressource

Le code de retour 200 signifie que l'opération s'est correctement passée, généralement c'est un code qui est associé à une requête de type GET.

Tous droits réservés à STUDI - Reproduction interdite

p. 14 Solution n°3

```python
from flask import Flask, request, jsonify
import json

app = Flask(__name__)

users = [
    {'id': 1, 'user': 'Jean'},
    {'id': 2, 'user': 'Arnaud'},
    {'id': 3, 'user': 'Charles'},
    {'id': 4, 'user': 'Marie'}
]

@app.route('/users', methods=['GET'])
def getUsersList():
    id = (request.args.get('id'))
    userName = users['id'==id]['user']
    return jsonify(userName)

@app.route('/users', methods=['DELETE'])
def removeUser():
    idToRemove = int(request.args.get('id'))
    for user in users:
        if (user['id'] == idToRemove):
            users.pop(idToRemove-1)
    return 'Success', 200

@app.route('/users', methods=['POST'])
def createNewUser():
    newUser = {'id' : 0, 'user' : 'username'}
    newUser['user'] = request.args.get('user')
    newUser['id'] = request.args.get('id')
    users.append(newUser)
    return 'Success', 201

if __name__ == '__main__':
   app.run(port=5000)
```

Vous remarquerez que la route /users appelée avec la méthode GET permet maintenant de spécifier, dans la requête, un 'id'. En spécifiant l'id, on récupère ainsi uniquement l'utilisateur correspondant à cet id.

Pour récupérer l'id entré par l'utilisateur dans la requête, on utilise toujours la fonction request.args.get().

Ensuite, on retourne toujours un objet Json (créé avec la fonction jsonify()).

Exercice p. 14 Solution n°4

Question 1

Quelle est la différence entre Websocket et API Rest ?

- ☐ Il n'y a pas de différence, c'est le même principe de fonctionnement
- ☑ Websocket n'est pas stateless
- ☑ Websocket est souvent plus rapide

Tous droits réservés à STUDI - Reproduction interdite

Les deux dernières propositions sont vraies. Les Websockets utilisent une connexion établie sur un lien TCP, qui requiert donc un handshake et une persistance de connexion. Websocket est souvent plus rapide, et utilisé pour des applications plus lourdes, comme par exemple du streaming vidéo.

Question 2

Qu'est ce qu'un '*endpoint*' ?

- ○ Une méthode HTTP
- ◉ Une adresse spécifique sur laquelle une requête est réalisée
- ○ Le nom d'un protocole de communication

Un endpoint spécifie l'adresse (url) utilisée pour réaliser une requête donnée. Souvent, les endpoints sont représentatifs des ressources mises à disposition par le serveur Rest.

Question 3

Postman permet de faire des requêtes http de type GET uniquement :

- ○ Vrai
- ◉ Faux

Postman est un outil qui permet de réaliser des requêtes http de tout type : GET, POST, DELETE, et encore beaucoup d'autres. Il est très utile dans la validation de notre serveur Rest.

Question 4

Que signifie la notion de mise en cache dans un serveur Rest ?

- ○ La possibilité de masquer les adresses IP lors d'un échange de données
- ◉ Il s'agit d'une caractéristique propre à l'architecture Rest, qui permet de donner une certaine persistance aux données récupérées du serveur
- ○ Il s'agit de la possibilité d'échanger des données sans maintenir une connexion persistante

La mise en cache permet de jouer sur la persistance de certaines données. C'est une notion qui est liée à l'aspect Stateless des API Rest.

Question 5

Flask est le seul framework permettant de réaliser des serveurs Rest.

- ○ Vrai
- ◉ Faux

Pour réaliser un serveur Rest, il faut pouvoir traiter des routes et des requêtes http. Flask est loin d'être la seule possibilité. Par exemple, Django permet de réaliser le même type d'opérations.

Tous droits réservés à STUDI - Reproduction interdite